

CORPORATE TRAINING · 1-HOUR HANDS-ON SESSION

Smart Data Validation with AI

*Prompt Engineering · Anomaly Detection · Automated Rule Generation & Quality Scoring***Built on the Databricks Lakehouse Platform**

Facilitator	Prachi
Format	60 minutes — concepts, live code, and a guided lab
Platform	Databricks (Notebooks, SQL, Foundation Model APIs, Unity Catalog)
Audience	Data engineers & analysts working with production data pipelines
Pre-requisites	Working knowledge of SQL and Python (PySpark helpful, not required)

Session Agenda

This is a working session. Concepts are kept tight so we spend most of the hour in the notebook. Every code block in this document runs on a standard Databricks cluster (DBR 14.x+ / Serverless) with Unity Catalog enabled and access to Foundation Model APIs.

Time	Module	What we cover
0:00 – 0:05	Framing the problem	Why classic data quality breaks; where AI adds leverage
0:05 – 0:20	1 · Prompt Engineering for Data Tasks	Patterns that make LLM output reliable enough for pipelines
0:20 – 0:35	2 · Anomaly Detection	Statistical + ML + LLM-assisted detection on Delta tables
0:35 – 0:55	3 · Smart Data Validation	Auto rule generation + a 6-dimension quality score
0:55 – 1:00	Wrap-up & lab	Putting it together; take-home exercise

The through-line

We move from *telling* the model what to check (prompting), to *finding* what we didn't know to check (anomaly detection), to *generating and scoring* the checks themselves (smart validation). Each module feeds the next.

Framing: Why AI-Assisted Data Quality

Traditional data quality is **rule-first**: an engineer hand-writes expectations — `not null`, `value in set`, `range 0-100`. This works, but it has three well-known failure modes:

- **Coverage gap** — rules only exist for problems someone already anticipated. Unknown-unknowns pass silently.
- **Maintenance drag** — schemas drift, business logic changes, and rule sets rot faster than anyone updates them.
- **Binary verdicts** — a row either passes or fails. There is no graded notion of “how healthy” a dataset is this morning versus yesterday.

AI addresses each of these — but only when applied deliberately. The mapping we use for the rest of the session:

Traditional gap	AI technique	Module
Rules are expensive to author	LLM generates candidate rules from schema + samples	3
Unknown-unknowns slip through	Statistical / ML anomaly detection	2
No graded health signal	Multi-dimension quality scoring	3
Rigid, brittle logic	Prompt-driven, adaptable checks	1

Guardrail we keep all session

AI proposes; deterministic code disposes. LLMs generate rules, explain anomalies, and draft scores — but the checks that gate a pipeline are always compiled down to plain, testable SQL / PySpark assertions. We never let a non-deterministic model be the final arbiter on a production row.

Module 1 · Prompt Engineering for Data Tasks

Goal: get LLM output that is **structured, deterministic-enough, and cheap** to run inside a data pipeline. Data tasks are unforgiving — a stray sentence of prose breaks a downstream parser. The patterns below are what make LLM calls production-safe.

1.1 Calling an LLM from Databricks SQL

Databricks exposes Foundation Models as SQL functions. `ai_query()` is the workhorse — it lets you score an entire Delta table with an LLM in one query, no Python loop.

```
SQL · ai_query() over a Delta table
-- Point ai_query at a pay-per-token foundation model endpoint.
-- Runs row-by-row across the whole table, in parallel, on the warehouse.
SELECT
  order_id,
  raw_address,
  ai_query(
    'databricks-meta-llama-3-3-70b-instruct',
    CONCAT(
      'Extract the country as a 2-letter ISO code from this address. ',
      'Respond with ONLY the code, nothing else. Address: ',
      raw_address
    )
  ) AS country_iso
FROM samples.orders
LIMIT 20;
```

Databricks also ships task-specific AI functions that wrap common patterns so you don't prompt from scratch:

Function	Use for	Returns
<code>ai_classify()</code>	Bucket a value into known labels	One of your labels
<code>ai_extract()</code>	Pull named entities / fields from text	Struct of fields
<code>ai_analyze_sentiment()</code>	Sentiment of free text	positive / negative / neutral
<code>ai_gen()</code>	General instruction-following generation	String
<code>ai_fix_grammar()</code>	Normalize / clean text fields	Cleaned string

```
SQL · built-in AI functions
-- Task-specific functions are shorter, cheaper, and more reliable
-- than hand-rolled prompts for the job they cover.
SELECT
  feedback_id,
  ai_classify(comment, ARRAY('billing','delivery','quality','other')) AS topic,
  ai_analyze_sentiment(comment) AS sentiment
FROM raw.customer_feedback;
```

1.2 Five prompt patterns that matter for data work

Pattern 1 — Force structured output (JSON)

Never parse prose. Demand JSON, specify the exact schema, and forbid extra text. On Databricks you can also pass a `response_format` to hard-constrain the shape.

```
Python · structured JSON output
from pyspark.sql.functions import expr
```

```
# response_format guarantees valid JSON matching your schema –
# no more "Sure! Here is the JSON:" preamble to strip.
prompt = """You are a data validator. For the customer record below,
return ONLY a JSON object with keys:
  email_valid (boolean), phone_valid (boolean), issues (array of strings).
Record: {record}"""

df = spark.sql("""
  SELECT customer_id,
    ai_query(
      'databricks-meta-llama-3-3-70b-instruct',
      CONCAT('Validate and return JSON {email_valid,phone_valid,issues}. Record: ',
        to_json(struct(email, phone))),
      responseFormat => 'JSON'
    ) AS validation
  FROM raw.customers
""")
df.select("customer_id", expr("from_json(validation, 'email_valid boolean, phone_valid boolean, issues
array<string>')").alias("v")).display()
```

Pattern 2 — Few-shot examples for edge cases

Show, don't just tell. Two or three worked examples pin down behaviour on the messy cases far better than a paragraph of instructions. This is the single highest-leverage move for classification-style data tasks.

```
Python · few-shot template
instruction = """Classify each product name into a category.
Examples:
  "iPhone 15 Pro 256GB"      -> Electronics
  "Organic Fuji Apples 1kg"  -> Grocery
  "1/4-20 Hex Bolt (100 pack)" -> Hardware
  "12mo Netflix Gift Card"   -> Digital
Now classify (respond with the category word only):
Product: {name}"""
```

Pattern 3 — Assign a role + constraints

- Role sets the frame: “You are a meticulous data-quality auditor.”
- Constraints kill ambiguity: “If a field is missing, treat it as invalid. Never guess. Never explain.”
- Output contract closes it: “Return only the JSON object.”

Pattern 4 — Decompose, don't overload

One prompt = one job. Asking a model to “clean, validate, categorise, and score” in a single call produces mush. Chain narrow calls (or narrow AI functions) instead — each is testable and cacheable.

Pattern 5 — Temperature & determinism

For validation you want repeatable answers. Set temperature to 0, and where possible constrain to a closed label set so the model can't get creative.

```
SQL · deterministic parameters
ai_query(
  'databricks-meta-llama-3-3-70b-instruct',
  prompt,
  modelParameters => named_struct('temperature', 0.0, 'max_tokens', 256)
)
```

Cost & scale note

`ai_query()` bills per token and runs per row. For a 50M-row table, don't LLM every row — sample, or gate the call behind a cheap deterministic pre-filter (e.g. only send rows a regex already flagged as suspicious). We use this “cheap filter first, LLM second” pattern again in Module 3.

Module 2 · Anomaly Detection

Prompting checks what we know to look for. Anomaly detection surfaces what we *don't*. We layer three approaches, cheapest first.

Layer	Method	Good at	Cost
Statistical	Z-score, IQR, MAD	Numeric outliers, fast, explainable	Very low
ML	Isolation Forest	Multivariate / subtle patterns	Medium
LLM-assisted	ai_query on flagged rows	Explaining & triaging anomalies	High

2.1 Statistical detection in PySpark

The MAD (Median Absolute Deviation) method is the practical default: it's robust to the very outliers you're hunting, unlike the mean/standard-deviation Z-score, which the outliers themselves distort.

```

Python · robust MAD outlier detection
from pyspark.sql import functions as F

def flag_outliers_mad(df, col, threshold=3.5):
    """Robust outlier flagging via Median Absolute Deviation.
    Modified z = 0.6745 * (x - median) / MAD ; |z| > 3.5 => outlier."""
    med = df.approxQuantile(col, [0.5], 0.001)[0]
    mad_df = df.withColumn("abs_dev", F.abs(F.col(col) - F.lit(med)))
    mad = mad_df.approxQuantile("abs_dev", [0.5], 0.001)[0] or 1e-9

    return df.withColumn(
        "mad_z",
        0.6745 * (F.col(col) - F.lit(med)) / F.lit(mad)
    ).withColumn(
        "is_outlier", F.abs(F.col("mad_z")) > threshold
    )

txn = spark.table("finance.transactions")
flagged = flag_outliers_mad(txn, "amount", threshold=3.5)
flagged.filter("is_outlier").orderBy(F.desc("mad_z")).display()

```

IQR is the other quick win — no distributional assumptions, ideal as a first pass in a Databricks SQL alert:

```

SQL · IQR outlier query
WITH bounds AS (
  SELECT
    percentile(amount, 0.25) AS q1,
    percentile(amount, 0.75) AS q3
  FROM finance.transactions
)
SELECT t.*
FROM finance.transactions t, bounds b
WHERE t.amount < b.q1 - 1.5 * (b.q3 - b.q1)
   OR t.amount > b.q3 + 1.5 * (b.q3 - b.q1);

```

2.2 Multivariate detection — Isolation Forest

Single-column stats miss anomalies that only show up across columns (a small amount is normal; a small amount from a brand-new account at 3 a.m. to a new payee is not). Isolation Forest isolates such points with few splits.

```

Python · Isolation Forest
from sklearn.ensemble import IsolationForest
import pandas as pd

```

```
# Pull a feature sample to the driver (or use pandas UDFs to scale out).
pdf = (spark.table("finance.transactions")
      .select("amount", "account_age_days", "hour_of_day", "n_prior_txns")
      .sample(fraction=0.1, seed=42)
      .toPandas())

iso = IsolationForest(
    n_estimators=200,
    contamination=0.02,  # expected anomaly rate – tune to your data
    random_state=42,
)
pdf["anomaly_score"] = iso.fit_predict(pdf)  # -1 = anomaly, 1 = normal
pdf["raw_score"]     = iso.decision_function(pdf) # lower = more anomalous

anomalies = pdf[pdf["anomaly_score"] == -1].sort_values("raw_score")
spark.createDataFrame(anomalies).write.mode("overwrite").saveAsTable("dq.txn_anomalies")
```

Scaling tip

For large tables, wrap the scoring in a `pandas_udf` or apply the model per-partition with `applyInPandas` so detection runs distributed across the cluster instead of collapsing to the driver.

2.3 LLM-assisted triage

Detection tells you a row is weird. The LLM tells you *why* — turning a raw anomaly list into something an analyst can action. Only send the already-flagged rows (cheap filter first).

SQL · LLM explanation of anomalies

```
-- Explain only the rows statistics already flagged – not the whole table.
SELECT
  txn_id, amount, account_age_days, hour_of_day,
  ai_query(
    'databricks-meta-llama-3-3-70b-instruct',
    CONCAT(
      'You are a fraud analyst. In one sentence, state the most likely ',
      'reason this transaction was flagged as anomalous. Data: ',
      to_json(struct(amount, account_age_days, hour_of_day, n_prior_txns))
    ),
    modelParameters => named_struct('temperature', 0.0)
  ) AS likely_reason
FROM dq.txn_anomalies;
```

Module 3 · Smart Data Validation

The centrepiece. We combine everything: an LLM **generates** candidate validation rules from a table's schema and a data sample; we compile those into deterministic checks; then we roll results into a **6-dimension quality score**.

3.1 Automated rule generation

Feed the model the schema plus a profiled sample and ask for machine-readable rules. Crucially, we ask for rules **as data (JSON)**, not as prose or code we'd have to trust blindly.

```
Python · generate rules from schema + sample
from pyspark.sql.functions import to_json, struct, collect_list

table = "sales.customers"
schema_str = spark.table(table).schema.simpleString()
sample = spark.table(table).limit(25).select(
    to_json(struct("*")).alias("row")
).agg(collect_list("row").alias("rows")).collect()[0]["rows"]

rule_prompt = f"""You are a senior data-quality engineer.
Given this table schema and sample rows, propose validation rules.
Return ONLY a JSON array. Each rule object must have:
    column (string), dimension (one of: completeness, validity,
    uniqueness, consistency, accuracy, timeliness),
    rule_type (not_null|unique|range|regex|allowed_values|freshness),
    params (object), severity (critical|warning).

Schema: {schema_str}
Sample rows: {sample}"""

rules_json = spark.sql(f"""
    SELECT ai_query('databricks-meta-llama-3-3-70b-instruct',
        '{rule_prompt.replace("'", "'')}',
        responseFormat => 'JSON') AS rules
""").collect()[0]["rules"]
```

A typical rule the model returns:

```
JSON · generated rule set (excerpt)
[
  {"column": "email", "dimension": "validity", "rule_type": "regex",
    "params": {"pattern": "^[@]+@[^@]+\\.[^@]+$"}, "severity": "critical"},
  {"column": "customer_id", "dimension": "uniqueness", "rule_type": "unique",
    "params": {}, "severity": "critical"},
  {"column": "signup_date", "dimension": "timeliness", "rule_type": "freshness",
    "params": {"max_age_days": 365}, "severity": "warning"},
  {"column": "country", "dimension": "validity", "rule_type": "allowed_values",
    "params": {"values": ["IN", "US", "UK", "AE", "SG"]}, "severity": "warning"}
]
```

Human-in-the-loop

Generated rules are **proposals**. A one-time review — approve, edit, reject — happens before they gate anything. Approved rules are versioned in a Delta table so the set is auditable and reproducible. The LLM saves the authoring effort, not the accountability.

3.2 Compiling rules into deterministic checks

Each JSON rule maps to a plain Spark expression. This is where non-determinism ends — every gate on production data is now ordinary, testable code.

```
Python · rule compiler
from pyspark.sql import functions as F

def compile_rule(df, rule):
    """Return a boolean column: True where the row PASSES the rule."""
    col, rt, p = rule["column"], rule["rule_type"], rule.get("params", {})
    c = F.col(col)
    if rt == "not_null":
        return c.isNull()
    if rt == "regex":
        return c.rlike(p["pattern"])
    if rt == "range":
        return (c >= p["min"]) & (c <= p["max"])
    if rt == "allowed_values":
        return c.isin(p["values"])
    if rt == "freshness":
        return F.datediff(F.current_date(), c) <= p["max_age_days"]
    if rt == "unique":
        w = F.count("*").over(Window.partitionBy(col)) # >1 => dup
        return w == 1
    return F.lit(True) # unknown rule types never block

def evaluate(df, rules):
    for i, r in enumerate(rules):
        df = df.withColumn(f"pass_{i}", compile_rule(df, r))
    return df
```

3.3 The quality score

A single pass/fail is uninformative. We score across the six classic data-quality dimensions, weight by severity, and produce a 0–100 index you can trend over time and alert on.

Dimension	Question it answers	Example metric
Completeness	Are required values present?	% non-null on key columns
Validity	Do values match format / domain?	% matching regex / allowed set
Uniqueness	Are there unwanted duplicates?	1 – (dup rows / total)
Consistency	Do related fields agree?	% passing cross-field checks
Accuracy	Do values reflect reality?	% matching reference source
Timeliness	Is the data fresh enough?	% within freshness window

```
Python · weighted 6-dimension quality score
from pyspark.sql import functions as F

SEV_WEIGHT = {"critical": 3.0, "warning": 1.0}

def quality_score(df, rules):
    scored = evaluate(df, rules)
    total = scored.count()
    dim_stats, weighted_num, weighted_den = {}, 0.0, 0.0

    for i, r in enumerate(rules):
        passed = scored.filter(F.col(f"pass_{i}")).count()
        pass_rate = passed / total if total else 1.0
        w = SEV_WEIGHT[r["severity"]]
        dim = r["dimension"]
```

```

    dim_stats.setdefault(dim, []).append(pass_rate)
    weighted_num += pass_rate * w
    weighted_den += w

    per_dimension = {d: round(100 * sum(v) / len(v), 1) for d, v in dim_stats.items()}
    overall = round(100 * weighted_num / weighted_den, 1) if weighted_den else 100.0
    return overall, per_dimension

overall, by_dim = quality_score(spark.table("sales.customers"), approved_rules)
print(f"Overall data-quality score: {overall}/100")
print("By dimension:", by_dim)

```

Persist each run so the score becomes a monitored time series — the graded health signal traditional DQ never gave us:

```

Python · persist score for trending & alerting
from datetime import datetime

result_row = [(datetime.now(), "sales.customers", overall,
                    by_dim.get("completeness"), by_dim.get("validity"),
                    by_dim.get("uniqueness"), by_dim.get("timeliness"))]
cols = ["run_ts", "table_name", "overall", "completeness", "validity", "uniqueness", "timeliness"]
(spark.createDataFrame(result_row, cols)
    .write.mode("append").saveAsTable("dq.quality_scores"))

# Then alert in Databricks SQL when the score drops:
# SELECT * FROM dq.quality_scores
# WHERE overall < 90 AND run_ts > current_date() - 1;

```

Putting It Together: The End-to-End Flow

The three modules form one pipeline. Read top to bottom:

1. **Profile** the target Delta table (schema + sample + basic stats).
2. **Generate** candidate rules with an LLM (Module 1 prompting + Module 3 generation).
3. **Review & version** the rules once; store approved rules in Delta.
4. **Detect anomalies** statistically / with Isolation Forest to catch what no rule covers (Module 2).
5. **Compile & evaluate** approved rules as deterministic Spark checks.
6. **Score** across six dimensions; persist and trend the 0–100 index.
7. **Alert & explain** — LLM triages the flagged rows into human-readable reasons.

Where this lives in Databricks

Wrap steps 1–7 in a notebook, schedule it as a **Databricks Workflow** per table (or trigger on new data via **Delta Live Tables**). Store rules and scores in Unity Catalog Delta tables. Surface the trend on a Databricks SQL dashboard, and route score-drop alerts to Slack/email via SQL Alerts.

Guided Lab & Take-Home Exercise

In-session (last 15 minutes)

- Point the rule generator at `samples.nyctaxi.trips` (or any table you own). Read the rules it proposes — do you agree?
- Run the MAD outlier function on `fare_amount`. How many outliers? Do the extreme ones make business sense?
- Compute the quality score. Which dimension is weakest?

Take-home

8. Add a new rule_type (e.g. cross-field consistency: `pickup_time < dropoff_time`) to the compiler.
9. Replace the driver-side Isolation Forest with an `applyInPandas` version so it scales.
10. Build a Databricks SQL dashboard trending the overall score for one table over a week.
11. Add a cheap deterministic pre-filter before any `ai_query` call and measure the token savings.

Key takeaways

- **AI proposes, deterministic code disposes** — LLMs draft rules and explanations; compiled checks gate the data.
- **Structure everything** — JSON output, closed label sets, temperature 0 for anything that must be repeatable.
- **Cheap filter first, LLM second** — never LLM a whole table when a regex or a statistic can pre-screen it.
- **Score, don't just gate** — a trended 6-dimension index turns data quality from a binary alarm into an observable signal.

Prepared by Prachi · Corporate Training · Smart Data Validation with AI on Databricks